

Graphlet dokumentáció

Tartalom

BEVEZETÉS	6
STACK.....	7
FELHASZNÁLT TECHNOLÓGIÁK (STACK) RÉSZLETES BEMUTATÁSA.....	7
KLIENSOLDALI (FRONTEND) TECHNOLÓGIÁK.....	7
<i>React.js</i>	7
<i>Saját fejlesztésű Gráf-Renderelő Motor (HTML5 Canvas)</i>	8
<i>Tailwind CSS és UI komponensek</i>	8
<i>Szerveroldali (Backend) Technológiák</i>	8
<i>C# Programozási Nyelv</i>	9
<i>ASP.NET Core Keretrendszer</i>	9
<i>JWT Autentikáció és Claims-alapú Autorizáció</i>	9
<i>Relációs Adatbázis és Adatintegritás</i>	10
<i>Entity Framework Core (EF Core)</i>	10
<i>Docker Konténerizáció</i>	10
ARCHITEKTÚRA	12
1.2.1. FELHASZNÁLÓI ESETEK (USE CASE) ÉS FUNKCIONÁLIS HATÁSKÖRÖK	12
<i>A Felhasználó (User)</i>	12
• Hitelesítési ág (Auth):	12
• Fő Vezérlőpult (MyWorkspaces):	12
• Munkaterület kezelés (Workspace):.....	13
• Beállítások (Settings):	13
1.2.2. KLIENSOLDALI ADATMODELL ÉS OSZTÁLYDIAGRAM (FRONTEND CLASSES)	13
<i>Entitások és Kapcsolataik:</i>	13
User és Public User:.....	13
Organization (Szervezet):	14
Workspace (Munkaterület):.....	14
A Gráf Alapjai: Note és NoteRelation:	14
1.2.3. RENDSZER KOMMUNIKÁCIÓS SZEKVENCIÁK (SEQUENCE DIAGRAM).....	15
1. HITELESÍTÉSI (AUTH).....	15
• Login:	15
• Logout:.....	15
2. MUNKATERÜLET (WORKSPACE) KEZELŐ	15
Listázás:	16
Létrehozás	16
Konkrét lekérés:	16
Frissítés (Átnevezés):.....	16
Törlés:	16
3. CÍMKE (TAG).....	16
Listázás:	16
Létrehozás:.....	16
Lekérés:	16
4. JEGYZET (NOTE / CSOMÓPONT).....	17
Listázás:	17
Létrehozás:.....	17
Megnyitás:.....	17
Frissítés:	17
Törlés:	17
5. KAPCSOLATOK ÉS HOZZÁRENDELÉSEK (NOTE-TAG ÉS NOTERELATION).....	17
• Címke és Jegyzet összerendelése:	17
• Jegyzetek közötti kapcsolatok (Relation) kezelése:	17

REST API	19
ARCHITEKTÚRA ÁTTEKINTÉS	19
Általános API-konvenciók	19
Tipikus HTTP státuszkódok:	19
Fő modulok	20
Autentikáció és session kezelés	20
Login flow	20
Frontend integráció (példa)	23
Best Practices	23
Összegzés	23
BACKEND TESZTEK	24
ÁTTEKINTÉS	24
TESZTFÁJLOK	24
A tesztek futtatása	24
TESZTELÉSI LEFEDETTSÉG	25
REST API (Graphlet.http)	25
WebSockets (WebSocket.http)	25
TESZTKÖRNYEZET	26
FRONTEND	32
BEVEZETÉS	32
FELÉPÍTÉS ÉS SZERKEZET	32
Mappa szinten	32
Kód szinten	32
App.tsx és bejelentkezés	32
Regisztráció	33
Workspaces oldal	33
WorkspacePrewiev	34
Other options	35
FRONTEND TESZTEK	37
AUTOMATA TESZTELÉS	37

MANUÁLIS TESZTELÉS

Manuális tesztek

Login

Teszt eset 1: A register oldalra való átirányítás sikeresen megtörtént, a login oldalra való visszatérés hibátlan volt, és minden navigációs funkció rendben működött.

Teszt eset 2: Üres adattal történő bejelentkezés során a rendszer hibátlanul jelezte a hiányzó adatokat, a hibaüzenet megjelenése sikeres volt, és a felhasználó megfelelően tájékoztatva lett.

Teszt eset 3: Üres email mezővel történő bejelentkezés sikeresen megtörtént, a rendszer hibátlanul jelezte a hiányzó emailt, és a hibaüzenet helyesen jelent meg.

Teszt eset 4: Üres jelszóval történő bejelentkezés során a rendszer hibátlanul jelezte a hiányzó jelszót, a hibaüzenet megjelenése rendben zajlott, és minden funkció megfelelően működött.

Teszt eset 5: Hibás adatokkal történő bejelentkezés sikeresen megtörtént, a rendszer helyesen jelezte az adatokat, a hibaüzenet megjelenése hibátlan volt, és minden visszajelzés pontosan jelent meg.

Teszt eset 6: Hibás email cím megadásakor a rendszer megfelelően jelezte a hibát, a hibaüzenet hibátlanul jelent meg, és a felhasználó minden esetben értesítést kapott a problémáról.

Teszt eset 7: Hibás jelszó megadásakor a rendszer rendben jelezte a hibát, a hibaüzenet megjelenése sikeres volt, és a felhasználói interakciók zavartalanul működtek.

Teszt eset 8: Helyes adatokkal történő bejelentkezés sikeresen megtörtént, a felhasználó a workspaces oldalon hibátlanul jelent meg, és minden navigációs és megjelenítési funkció rendben működött.

Register

Teszt eset 1: A login oldalra való visszatérés sikeresen megtörtént, minden navigáció hibátlanul működött, és a felhasználó rendben visszakerült a kezdőoldalra.

Teszt eset 2: Üres adatokkal történő regisztráció során a rendszer hibátlanul jelezte a hiányzó mezőket, a hibaüzenetek rendben jelentek meg, és a felhasználó értesítése sikeres volt.

Teszt eset 3: Üres email mezővel történő regisztráció során a rendszer hibátlanul jelezte a hiányzó emailt, a hibaüzenet rendben megjelent, és minden funkció megfelelően működött.

.....	40
FELHASZNÁLÓI DOKUMENTÁCIÓ	41
A PROBLÉMA LEÍRÁSA ÉS A SZOFTVER CÉLJA	41

<i>A probléma:</i>	41
<i>A Graphlet megoldása:</i>	41
<i>2.2. Szerepkörök bemutatása</i>	41
1. Regisztrált Felhasználó (User)	41
2. Szervezeti Adminisztrátor (Organization Admin / Owner)	42
3. Publikus / Meghívott Felhasználó (Public User / Guest)	42
4. Eset: Kapcsolatok (Relations) kialakítása	45
ÖSSZEFOGLALÁS	47
KITEKINTÉS	48
IRODALMI JEGYZÉK	49

Bevezetés

A Graphlet egy webalapú alkalmazás, amelyet a Sulis vizsgaremek projekt keretében fejlesztettünk. A projekt célja egy modern, könnyen üzembe helyezhető rendszer biztosítása, amely ASP.NET Core alapú backendet és React frontend technológiát ötvözt.

A rendszer teljes mértékben konténerizált: Docker Compose segítségével egyetlen paranccsal elindítható a teljes alkalmazáskörnyezet, így nincs szükség manuális függőségkezelésre vagy bonyolult konfigurációra. A backend C# nyelven íródott (~65%), a frontend TypeScript és React kombináción alapul (~30%), a megjelenést pedig CSS stíluslapok egészítik ki.

Ez a dokumentáció bemutatja az alkalmazás architektúráját, az egyes komponensek működését, az API-végpontokat, valamint az üzembe helyezés és fejlesztés lépéseit. Akár fejlesztőként, akár felhasználóként érkezel, itt megtalálod a projekt megértéséhez szükséges összes információt.

Stack

Felhasznált technológiák részletes bemutatása

A Graphlet projekt technológiai stack-jének kiválasztása egy hosszas tervezési és kutatási folyamat eredménye. Egy modern, webalapú jegyzet-szerkesztő alkalmazás speciális kihívásokkal néz szembe: a frontend oldalon rendkívül gyors és folyamatos DOM-manipulációra van szükség a csomópontok mozgatása (drag-and-drop és pan) során, míg a backend oldalon a strukturált (relációs) és a strukturálatlan (metaadatok) információk hatékony, biztonságos és gyors kiszolgálása a cél. Ennek megfelelően a projekt egy robusztus, modernizált, JavaScript/TypeScript alapú ökoszisztémára épít a kliens oldalon, míg a backend ASP.NET keretrendszerben íródott, ami

Kliensoldali (Frontend) Technológiák

A kliensoldal az alkalmazás "arca", ahol a felhasználói interakciók döntő többsége történik. A cél egy reaktív, reszponzív és magas teljesítményű Single Page Application (SPA) létrehozása volt.

React.js

A frontend alkalmazás alapját a Meta (korábban Facebook) által fejlesztett React.js könyvtár adja. A React komponens-alapú architektúrája tökéletesen illeszkedik a Graphlet moduláris felépítéséhez. A UI különálló, újrahasznosítható darabokból (komponensekből) épül fel, mint például a csomópontok, a tulajdonságlapok, a navigációs sáv vagy az űrlapok. A React 18-ban bevezetett Concurrent Rendering és a Virtual DOM technológia elengedhetetlen egy olyan alkalmazásnál, ahol tucatnyi vagy akár száznyi gráf-elemet kell valós időben újrenderelni anélkül, hogy a böngésző szálát (main thread) megakasztanánk, és ezzel rontanánk a felhasználói élményt (frame rate drop).

A Frontend TypeScript-ben íródott. Bár a sima JavaScript gyorsabb prototipizálást tenne lehetővé, egy gráfokat kezelő rendszernél az adatstruktúrák (például a Node és Edge objektumok koordinátákkal, metaadatokkal és hivatkozásokkal teli felépítése) komplexek. A TypeScript interfészek (Interfaces) és típusdefiníciók (Types)

használatával a fejlesztési idő alatt (compile-time) kiszűrhető a hibákigen nagy része. Továbbá a modern integrált fejlesztőkörnyezetek (IDE-k), mint a VS Code, a TypeScript révén kiváló kódkiegészítést (IntelliSense) és refaktorálási eszközöket biztosítanak, ami egy több fejlesztős projektnél felbecsülhetetlen érték.

Saját fejlesztésű Gráf-Renderelő Motor (HTML5 Canvas)

A piacon elérhető dobozos gráf-megjelenítő könyvtárak sok esetben komoly teljesítménybeli korlátokkal vagy túlságosan kötött API-val rendelkeznek. Emiatt a projekt egy rugalmasabb, natív webes technológiákra (HTML5 Canvas) épülő megjelenítő réteget alkalmaz a gráfok kirajzolására. Ez a megoldás maximális kontrollt biztosít a fejlesztők számára a renderelési ciklus felett. A csomópontok (Nodes) és élek (Edges) pozíciójának kiszámítása, a drag-and-drop (fogd és vidd) események lekezelése, valamint a nagyítás/kicsinyítés (zoom/pan) natív eseménykezelőkön (Event Listeners) keresztül történik.

A gráf aktuális állapotának (hol helyezkednek el a csomópontok, melyik van éppen kijelölve, mik a tulajdonságaik) memóriában tartása a React beépített állapotkezelőivel lett megoldva (useState, Context API).

Tailwind CSS és UI komponensek

Az alkalmazás stílusozásához a Tailwind CSS utility-first keretrendszert integráltuk. Szemben a hagyományos (pl. Bootstrap vagy egyedi BEM-alapú SASS) megoldásokkal, a Tailwind lehetővé teszi, hogy a CSS osztályokat közvetlenül a JSX kódba írjuk. Ez felgyorsítja a fejlesztést és csökkenti a globális CSS fájlok méretét (a build folyamat végén a PurgeCSS eltávolítja a nem használt osztályokat). A Tailwind segítségével a reszponzív design (mobil és asztali nézetek) implementálása csupán néhány konfigurációs lépést igényelt.

Szerveroldali (Backend) Technológiák

A backend feladata a komplex üzleti logika végrehajtása, a jogosultságok szigorú ellenőrzése, az adatok validálása és az adatbázissal való biztonságos kommunikáció. Ehhez a projekt a Microsoft által fejlesztett, modern és vállalati szintű (enterprise-grade) technológiákat alkalmazza.

C# Programozási Nyelv

A szerveroldali üzleti logika teljes egészében C# nyelven íródott. A C# egy modern, objektumorientált, és erősen típusos nyelv, amely kiválóan alkalmas komplex, nagyvállalati szintű szoftverek fejlesztésére. A szigorú típusosság garantálja, hogy az adatszerkezetek (például egy gráf csomópontjának reprezentációja) már fordítási időben (compile-time) validálva legyenek, minimálisra csökkentve a futásidejű hibák (runtime errors) esélyét. Emellett a nyelv olyan fejlett funkciói, mint a LINQ (Language Integrated Query) és az aszinkron programozási modell (async/await), rendkívül olvashatóvá és karbantarthatóvá teszik az adatok szűrését és feldolgozását.

ASP.NET Core Keretrendszer

A REST API végpontok kiszolgálását az ASP.NET Core keretrendszer végzi. Ez egy nyílt forráskódú, keresztplatformos (cross-platform), és rendkívül magas teljesítményű keretrendszer. Az ASP.NET Core beépített, első osztályú Dependency Injection (Függőség-injektálás) tárolóval rendelkezik, amely elősegíti a laza csatolást (loose coupling) az egyes szoftverkomponensek (Kontrollerek, Szolgáltatások, Repozitóriumok) között. Ez a moduláris felépítés nemcsak az olvashatóságot javítja, hanem a backend kód egységtesztelését (unit testing) is egyszerűbbé teszi a függőségek mockolásán keresztül. A HTTP kérések feldolgozása egy testeszable middleware-en halad keresztül, amely hatékony hiba- és naplókezelést tesz lehetővé.

JWT Autentikáció és Claims-alapú Autorizáció

Az adatbiztonság és a hozzáférés-vezérlés kulcsfontosságú a gráfok védelme érdekében. A rendszer állapotmentes (stateless) autentikációt használ JSON Web Token (JWT) formájában. Az ASP.NET Core beépített hitelesítési middleware-jét (JWT Bearer Authentication) használjuk a beérkező HTTP kérések fejlécében lévő tokenek validálására. Az autorizáció Claims-alapú (szerepkörök és jogosultságok alapján történik). Az egyes API végpontok (például a gráf törlése) felett az [Authorize] attribútum és a hozzáférési házirendek (Policies) garantálják, hogy csak a megfelelő jogosultsággal rendelkező felhasználó hajthassa végre a műveletet, megakadályozva az illetéktelen adathozzáférést.

Az adatok tartós tárolása, az adatintegritás megőrzése és a rendszer üzemeltetése modern szoftverfejlesztési elveket követ.

Relációs Adatbázis és Adatintegritás

Az alkalmazás elsődleges adattára egy relációs adatbázis-kezelő (RDBMS). Mivel a gráfok adatai (Felhasználók, Projektek/Gráfok metaadatai, Csomópontok kapcsolatai) szigorúan strukturáltak, az adatintegritás (ACID tulajdonságok) és az idegen kulcsok (Foreign Keys) használata kötelező. A rendszer úgy lett kialakítva, hogy a csomópontok (Nodes) és élek (Edges) közötti relációk a lehető leggyorsabban lekérdezhetőek legyenek, miközben az esetlegesen dinamikus metaadatok (pl. csomópontok egyedi stílusai) szerializált formában is biztonságosan eltárolhatók.

Entity Framework Core (EF Core)

Az adatbázissal való kommunikációt nem nyers SQL lekérdezésekkel, hanem az Entity Framework Core (EF Core) ORM (Object-Relational Mapping) eszközzel végezzük. Az EF Core lehetővé teszi a fejlesztők számára, hogy az adatbázis tábláit C# osztályokként (Entitásokként) kezeljék. A projekt a "Code-First" (kód-először) megközelítést alkalmazza: az adatbázis sémáját és kapcsolatait a C# kód határozza meg, amelyből az EF Core automatikusan generálja le a verziókövetett adatbázis-migrációkat. Ez biztosítja, hogy az adatbázis szerkezete és az alkalmazás kódja mindig szinkronban legyen. A komplex adatbázis-lekérdezéseket a LINQ segítségével, típusbiztos módon írjuk meg, így a szintaktikai hibákat már a kód írásakor jelezni tudja a fejlesztőkörnyezet.

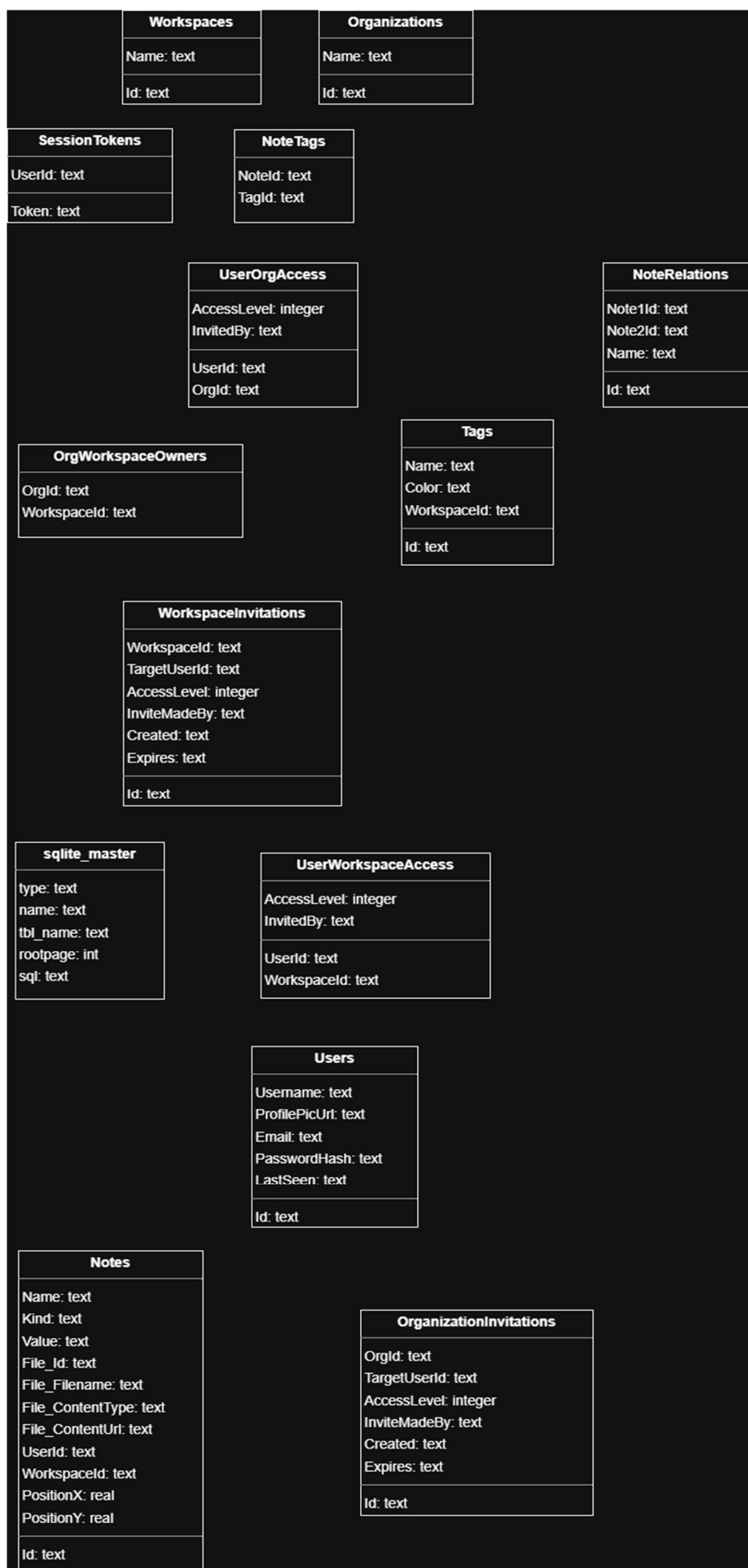
Docker Konténerizáció

A platformfüggetlenség és a konzisztens fejlesztői, valamint éles környezetek biztosítása érdekében a projekt támogatja a Docker alapú konténerizációt. Az ASP.NET Core alkalmazás egy optimalizált, könnyűsúlyú Linux-alapú Docker image-ben fut, ahogy a frontend is. A mikroszolgáltatásos és konténeres architektúra nemcsak a helyi fejlesztést teszi egyszerűbbé (például a docker-compose használatával az adatbázis és a backend egy gombnyomásra indítható), hanem a későbbi felhő alapú (Cloud) skálázhatóságot is maximálisan kiszolgálja.

Az

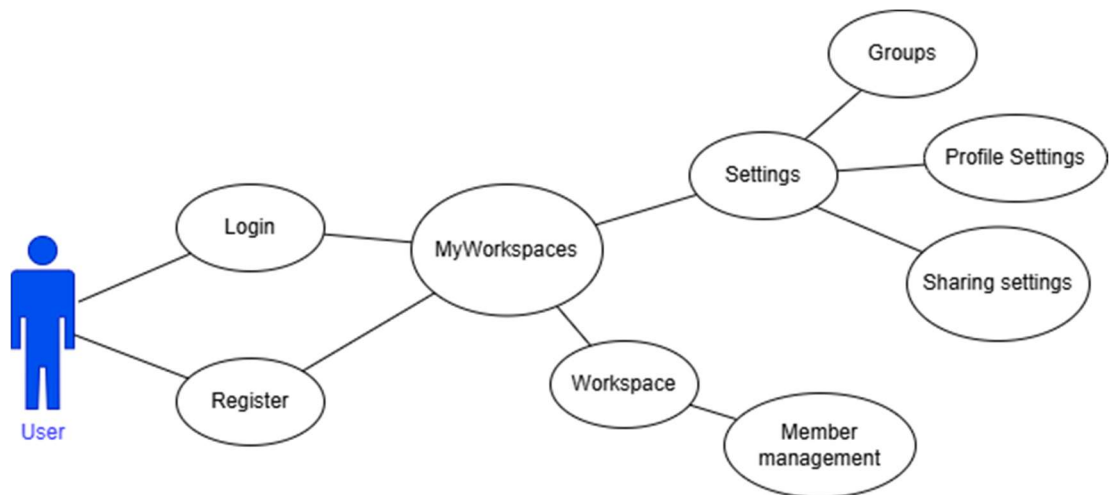
adatbázis

felépítése:



Architektúra

A Graphlet alkalmazás architekturális tervezése során a modularitás, a tiszta felelősségi körök és a skálázhatóság voltak a fő szempontok. A rendszer működését, az adatmodelleket és a kliens-szerver interakciókat az alábbi diagramok és elemzések mutatják be részletesen.



1.2.1. Felhasználói Esetek (Use Case) és Funkcionális Hatáskörök

A Usecase.drawio diagram a rendszer magas szintű funkcionális felépítését és a végfelhasználó (User) interakciós lehetőségeit modellezi. A diagram egy fa-struktúrájú navigációs és jogosultsági hierarchiát rajzol ki.

A Felhasználó (User)

A rendszer egyetlen elsődleges aktort definiál: a regisztrált Felhasználót. A felhasználó az alkalmazásba lépve a következő ágakon navigálhat végig:

- **Hitelesítési ág (Auth):**

A belépési pont. A felhasználó a Login (Bejelentkezés) vagy Register (Regisztráció) folyamatokon keresztül azonosítja magát. Sikeres hitelesítés után a rendszer a fő vezérlőpultra irányítja.

- **Fő Vezérlőpult (MyWorkspaces):**

Ez az alkalmazás központi csomópontja (hub-ja). Innen ágazik szét a felhasználó privát és megosztott munkaterületeinek kezelése, valamint a profilbeállítások.

- **Munkaterület kezelés (Workspace):**

A MyWorkspaces-ből megnyitható egy konkrét munkaterület. Ehhez szorosan kapcsolódik a Member management (Tagok kezelése) funkció, amely utal arra, hogy a munkaterületek kollaboratívak. Itt hívhatók meg új tagok (például olvasói vagy szerkesztői jogosultsággal), illetve itt távolíthatók el a korábbi résztvevők.

- **Beállítások (Settings):**

- A vezérlőpultról elérhető globális beállítások három fő területre bomlanak:
- Profile Settings: A felhasználó saját fiókadatai (név, jelszó, avatar).
- Sharing settings: A megosztási alapértelmezések és házirendek kezelése.
- Groups: Csoportok kezelése, amely megkönnyíti a tömeges jogosultság-kiosztást (például egy teljes fejlesztői csapat hozzáadását egy munkaterülethez).

1.2.2. Kliensoldali Adatmodell és Osztálydiagram (Frontend Classes)

A Frontend_classes.drawio diagram az alkalmazás domein modelljét írja le, amely a frontend állapotkezelésének és az ASP.NET backend adatbázis-entitásainak is az alapját képezi. A rendszer egyértelműen a "Munkaterület" (Workspace) és a "Jegyzet/Csomópont" (Note) koncepció köré épül.

Entitások és Kapcsolataik:

User és Public User:

- User: A hitelesített felhasználó teljes adatlapja. Tartalmazza a privát email címet, a profilkép URL-jét (picurl), egy egyedi azonosítót (Id: uuid), valamint egy utolsó aktivitást jelző bélyeget (lastSeen: DateTime).
- Public User: Ez az osztály egy Data Transfer Object (DTO) mintát követ. Amikor más felhasználók adatait kérjük le (például a "Member management" felületen), a rendszer csak publikus adatokat ad vissza (Név,

email, picurl), elrejtve a szenzitív belső azonosítókat és státuszokat. Ebből látszik a "Name" mező megjelenése, ami a belső User modellben nincs explicit kiemelve.

Organization (Szervezet):

- Több-bérlős (Multi-tenant) struktúrára utaló entitás. Rendelkezik azonosítóval (id), névvel (name), egy opcionális taglistával (members: List<access>), és birtokolhat több munkaterületet (workspaces: List<>). Ez teszi lehetővé a B2B felhasználást, ahol egy cég fiókja alá tartozik a dolgozók összes munkaterülete.

Workspace (Munkaterület):

- A gráfok és jegyzetek logikai konténere. Rendelkezik id-val és name-mel. A User entitás hivatkozik rá, jelezve a tulajdonosi vagy hozzáférési viszonyt.

A Gráf Alapjai: Note és NoteRelation:**Note (Jegyzet):**

Ez a vásznon lévő legfontosabb elem. Rendelkezik címmel (name/title), címkék listájával (List<tag>), és a kapcsolatok listájával (List<NoteRelation>). Tartalmaz egy típust (Enum: text, url, file), amely meghatározza a Value (Érték) viselkedését. Ha a típus fájl, akkor egy opcionális FileInfo osztályt használ.

NoteRelation (Kapcsolat):

Ez az osztály köti össze a gráf pontjait. Egyértelműen tárolja a két végpont azonosítóját (id(note1.Id, note2.Id)) és a kapcsolat nevét/típusát (name).

Tag (Címke):

Metaadatok a Note-ok kategorizálásához. Egyedi id-val, névvel és színnel (color) rendelkezik a vizuális elkülönítéshez.

FileInfo (Fájlkezelés):

A csatolmányok metaadatait tárolja: id, fájlnev (filename), kiterjesztés típusa (contentType enum: image, audio, video, other), és az elérési út (contenturl).

RenderFrame (A Vizuális Vásznon):

Egy speciális, a diagramon "Experimental idea" (Kísérleti ötlet) címkével ellátott osztály. Ez a frontend renderelő motorjának az állapotát (State) reprezentálja. Tartalmazza az aktuálisan megjelenített jegyzetek listáját (List<Note>), az aktív zárolásokat a kollaborációhoz (Locks: uuid), és az éppen kiválasztott elem azonosítóját (selectedElement: uuid).

1.2.3. Rendszer Kommunikációs Szekvenciák (Sequence Diagram)

A SequenceDiagram.drawio.pdf az alkalmazás legtechnikaibb dokumentuma, amely az egyes funkciók végrehajtása során a User (Felhasználó), az App (Kliens/Frontend) és az API (ASP.NET Backend) közötti pontos HTTP folyamatokat írja le. Az architektúra egyértelműen szigorú, szabványos RESTful elveket követ.

A diagram a következőkben taglalt modulokra bontható, amelyek bemutatják a HTTP metódusok, az útvonalak (útválasztás) és a szabványos válaszkódok (HTTP Status Codes) használatát.

1. Hitelesítési (Auth)

A belépés és kilépés folyamata:

- **Login:**
 - Az App egy POST /api/login kérést küld az API-nak.
 - Sikeres esetben az API 200 - OK (valószínűleg JWT tokennel) tér vissza.
 - Helytelen adatoknál 400 - Unauthorized a válasz (Megjegyzés: a 401 a szabványos Unauthorized, a 400 Bad Request, a dokumentáció a diagramon itt egyedi kezelést vagy elírást tartalmazhat).
- **Logout:**
 - A kijelentkezés a POST /api/logout végponton történik, 200 - OK sikerességgel.

2. Munkaterület (Workspace) Kezelő

A rendszer alapvető CRUD (Create, Read, Update, Delete) operációkat biztosít a munkaterületekhez:

Listázás:

GET /api/workspace -> Visszaadja JSON formátumban az összes elérhető munkaterületet (200 OK). Ha a token lejárt, 401 Unauthorized hibát dob.

Létrehozás:

POST /api/workspace -> A kérés JSON body-t vár. Sikeres generálás esetén 200 OK - json a válasz.

Konkrét lekérés:

GET /api/workspace/{id} -> Lekéri egy adott workspace adatait. Hiba esetén 404 Not Found választ ad, ha az UUID nem létezik.

Frissítés (Átnevezés):

PUT /api/workspace/{id} -> Paraméterként a JSON-ben az UUID-t és a string name-et (új név) várja. Válasz lehet 400 Bad Request (pl. üres név) vagy 404 Not found.

Törlés:

DELETE /api/workspace/{id} -> Siker esetén a szabványos 204 - No content kóddal tér vissza, jelezve, hogy a törlés megtörtént, nincs visszaadandó adat.

3. Címke (Tag)

A címkék a munkaterületekhez (Workspace) kötődnek szorosan:

Listázás:

GET /api/workspace/{workspaceId}/tags

Létrehozás:

POST /api/workspace/{workspaceId}/tags -> Body-ban várja a color és name paramétereket. Sikeres mentésnél 201 - Created kódot ad vissza, ami tökéletes RESTful gyakorlat új erőforrás létrehozásánál.

Lekérés:

GET /api/workspace/{workspaceId}/tags/{tagId} -> Egy specifikus címke adatainak lekérése.

4. Jegyzet (Note / Csomópont)

Ez a szekció kezeli a gráf tényleges tartalmát:

Listázás:

GET /api/note

Létrehozás:

POST /api/note -> 201 - Created a sikeres válasz.

Megnyitás:

A diagram mutat egy POST /api/note/{noteId} folyamatot "Opens a workspace/Note" felirattal, ami valószínűleg egy állapot-frissítő, rögzítő, vagy szeparált lekérő funkció (hagyományosan a GET felel a lekérésért).

Frissítés:

GET /api/note/{noteId} (Itt a diagram az UPDATE note ág alatt egy GET kérést specifikál frissítésként a 3. oldalon, de hagyományosan ez PUT/PATCH lenne. Feltételezhető, hogy a folyamat egy lekéréssel kezdődik a frissítés előtt).

Törlés:

DELETE /api/note/{noteId} -> A sikeres választ szintén a szabványos 204 - No content reprezentálja.

5. Kapcsolatok és Hozzárendelések (Note-Tag és NoteRelation)

A legkomplexebb műveletek, amelyek a vizuális hálót (gráfot) alkotják:

- **Címke és Jegyzet összerendelése:**
 - Lekérdezés: GET /api/note/{noteId}/tags/{tagId}
 - Hozzárendelés (Attach): PUT /api/note/{noteId}/tags/{tagId} -> A PUT metódus itt az idempotens kapcsolat-létrehozást szolgálja.
 - Eltávolítás (Detach): DELETE /api/note/{noteId}/tags/{tagId}
- **Jegyzetek közötti kapcsolatok (Relation) kezelése:**
 - Létrehozás: POST /api/note/{noteId}/relation -> 201 - Created. Létrehoz egy élt két csomópont között.

- Lekérdezés: GET /api/note/{noteId}/relation/{relationId}
- Szerkesztés (Frissítés): PUT /api/note/{noteId}/relation/{relationId} -> Lehetőséget ad a kapcsolat metaadatainak (például a kapcsolat neve) frissítésére (200 OK).
- Törlés: DELETE /api/note/{noteId}/relation/{relationId} -> Élek törlése a vászonról (204 - No content).

Rest API

Ez a dokumentum a Graphlet rendszer REST API-jának hivatalos, fejlesztői (corporate) dokumentációja. Célja, hogy részletesen bemutassa a rendszer működését, a frontend–backend kommunikációt, valamint az endpointok használatát valós use case-ek mentén.

Specifikáció: OpenAPI 3.1.1

Architektúra áttekintés

Általános API-konvenciók

- Alap URL (lokálisan): `http://localhost:<backend_port>/api`
- JSON formátumú kérés- és válasz-törzsek.
- HTTP metódusok:
- GET – adatok lekérdezése.
- POST – új erőforrás létrehozása, vagy művelet kezdeményezése (pl. login).
- PUT/PATCH – létező erőforrás frissítése.
- DELETE – erőforrás törlése.

Tipikus HTTP státuszkódok:

- 200 OK – sikeres kérés.
- 201 Created – erőforrás létrehozva.
- 400 Bad Request – hibás bemenet, validációs hiba.
- 401 Unauthorized – nem hitelesített kérés.
- 403 Forbidden – jogosultság hiánya.
- 404 Not Found – nem létező erőforrás.
- 500 Internal Server Error – váratlan hiba.

A rendszer klasszikus client-server modellben működik:

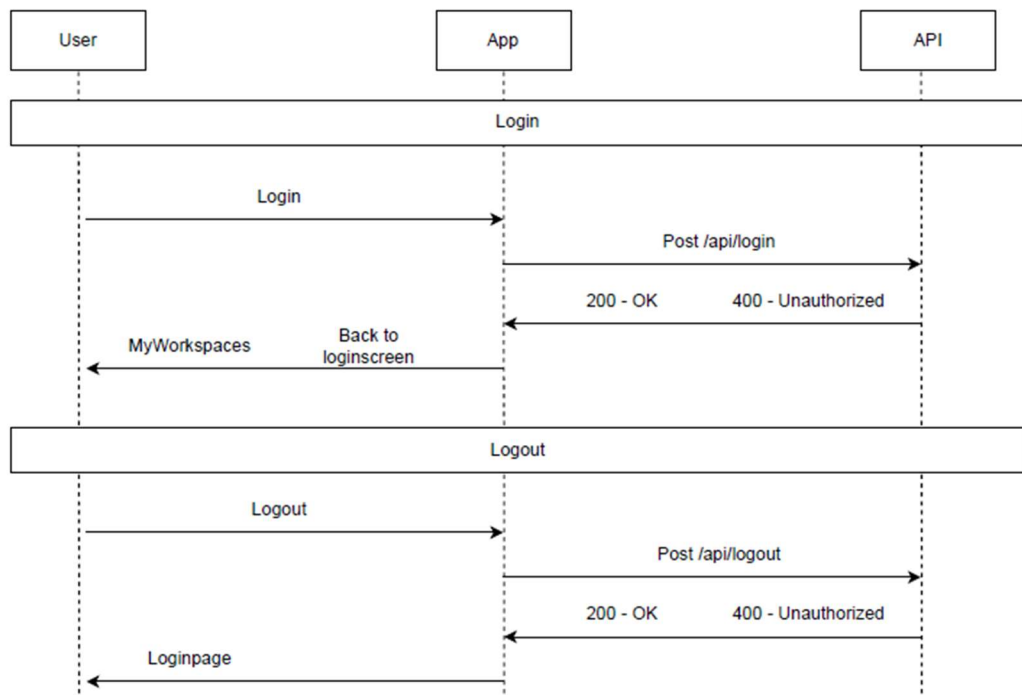
- Frontend (Web App)
- Backend (REST API)
- Adatbázis

Fő modulok

- Auth
- User
- Organization
- Workspace
- Note
- Tag
- Access Control

Autentikáció és session kezelés

Login flow



Endpoint

POST /api/login

Request:

```
{  
  "email": "user@email.com",  
  "password": "password"  
}
```

Response:

- 200 OK
- 401 Unauthorized

Workspace létrehozás

POST /api/workspace

Request:

```
{  
  "name": "My Workspace"  
}
```

Response:

```
{  
  "id": "uuid",  
  "name": "My Workspace"  
}
```

Hibák:

- 400 – invalid name
- 401 – unauthorized

6.2 Note létrehozás

POST /api/workspace/{workspaceId}/note

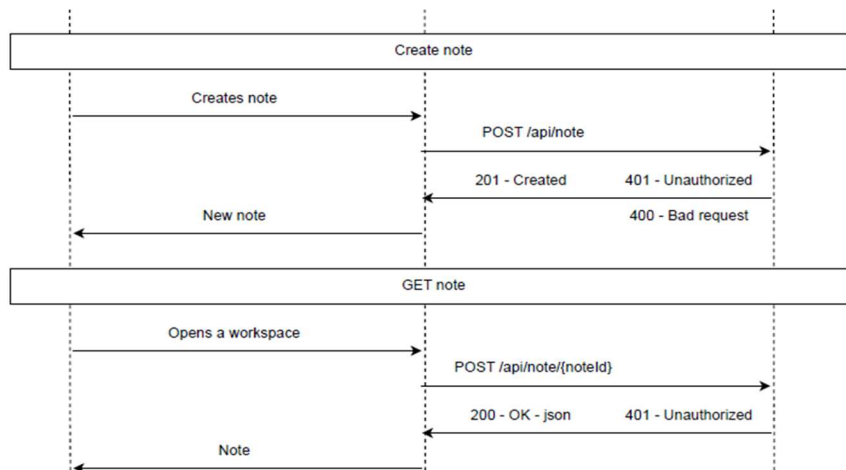
```
{  
  "name": "Note 1",  
}
```

```

"kind": "text",
"value": "Hello",
"positionX": 0,
"positionY": 0
}

```

Sequence:



6.3 Access grant

POST /api/access/workspace/{workspaceId}/grant

```

{
  "targetUserId": "uuid",
  "accessLevel": "admin"
}

```

6.4 Tag kezelés

POST /api/workspace/{workspaceId}/tags

```

{
  "name": "Important",
  "color": [255,0,0]
}

```

Példa

```

{
  "error": "Validation failed",

```

```
"details": {  
  "name": "Required"  
}  
}
```

Frontend integráció (példa)

```
await fetch('/api/workspace', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ name: 'Test' })  
})
```

Best Practices

- Minden request esetén történik validáció
- Idempotens endpointok használata
- HTTP status kódok konzisztens kezelése

Összegzés

A Graphlet API egy moduláris, skálázható REST architektúra, amely támogatja a collaborative workflow-kat, granular access control-lal és jól definiált resource modellel.

Backend tesztek

Áttekintés

A Grahplet backend JetBrains HTTP Client .http fájlokat használ integrációs és regressziós teszteléshez. Ezek a fájlok olyan kéréseket tartalmaznak, amelyek egy

futó API példány ellen futtathatók, beépített JavaScript tesztekkel a válaszok ellenőrzéséhez.

Tesztfájlok

Két elsődleges tesztfájl található a “Grahplet/” mappában:

1. Grahplet.http : A REST API végpontokat fedi le, beleértve a hitelesítést, a felhasználókezelést, a munkaterületeket (workspaces), címkéket (tags), jegyzeteket (notes) és a hozzáférés-kezelést.

2. WebSocket.http: A WebSocket-en keresztüli valós idejű együttműködési funkciókat fedi le, beleértve a hitelesítést, a munkamenet-információkat (session info), a zárolást/feloldást (locking/unlocking) és a “Heartbeat” (Ping/Pong).

A tesztek futtatása

JetBrains Rider / IntelliJ IDEA használatával (Ajánlott)

1. Indítsa el a Grahplet backendet (például a Grahplet projekt futtatásával).
2. Nyissa meg a Grahplet.http vagy a WebSocket.http fájlt a szerkesztőben.
3. Kattintson a “Run all requests in file” in file ikonra (dupla zöld nyíl) a szerkesztő margóján, vagy kattintson a futtatás ikonra az egyes kérések mellett.
4. Ellenőrizze az eredményeket a Services eszköztáblakban a “Test Results” fül alatt.

Parancssor használatával (Folyamatos integráció)

A .http tesztek a ijhttp eszközzel futtathatók:

Az eszköz telepítése (Docker vagy manuális telepítés szükséges) Lásd: <https://www.jetbrains.com/help/rider/http-client-cli.html>

REST API tesztek futtatása: “ijhttp Grahplet/Grahplet.http --env-file Grahplet/http-client.env.json”

WebSocket tesztek futtatása “ijhttp Grahplet/WebSocket.http” (Megjegyzés: Győződjön meg róla, hogy a backend fut és elérhető a fájlokban meghatározott baseUrl címen a CLI eszköz futtatása előtt.)

Tesztelési lefedettség

REST API (Grahplet.http)

- Hitelesítés: Bejelentkezés (sikeres/sikertelen), Kijelentkezés és Regisztráció.
- Felhasználókezelés: Aktuális felhasználó lekérése (/me), publikus profilok és felhasználói adatok frissítése.
- Munkaterületek (Workspaces): CRUD műveletek munkaterületeken.
- Címkék (Tags): Címkék létrehozása, listázása, frissítése és törlése egy munkaterületen belül.
- Jegyzetek (Notes):
 - Jegyzetek létrehozása (szöveges és URL típusok).
 - Jegyzetek frissítése és törlése.
 - Címkék hozzárendelése/leválasztása jegyzetekhez.
 - Jegyzetek közötti kapcsolatok kezelése.
- Szervezetek (Organizations): CRUD műveletek szervezeteken és a szervezeti munkaterületek listázása.
- Hozzáférés-kezelés
 - Munkaterület és Szervezet hozzáférési szintek (Olvasás/Írás/Admin).
 - Hozzáférés megadása, frissítése, visszavonása és meghívók kezelése.

WebSockets (WebSocket.http)

- Auth: Kapcsolat ellenőrzése érvényes tokennel és munkaterület-azonosítóval.
- Munkamenet állapota (Session State): A SessionInfo (aktív felhasználók és aktuális zárolások) fogadásának ellenőrzése.
- Heartbeat: Automatikus Ping/Pong ciklus ellenőrzése a kapcsolat életben tartásához.
- Valós idejű együttműködés:
 - Jegyzet zárolása szerkesztéshez.
 - Jegyzet feloldása.
 - A műveletek helyes továbbításának (broadcast) ellenőrzése.

Tesztkörnyezet

A tesztek alapértelmezés szerint a `http://localhost:5188` címre vannak konfigurálva. Az olyan változók, mint a `@token`, `@workspaceId` és `@noteId`

automatikusan rögzítésre kerülnek és megosztásra kerülnek a kérések között a "Run All" futtatás során, hogy megkönnyítsék az összetett folyamatokat (pl.

munkaterület létrehozása, majd egy jegyzet hozzáadása ahhoz).

Adatok tisztítása

Jelenleg a tesztek DELETE kéréseket tartalmaznak a folyamatok végén a létrehozott entitások (jegyzetek, címkék, munkaterületek, szervezetek) törléséhez, ha egymás után futtatják őket.

Teszt neve	Leírás (magyarul)
Grahplet.http	
Login	Bejelentkezés demo adatokkal.
Register New User	Új felhasználó regisztrációja egyedi felhasználónévvel és emaillel.

Login with New User	Bejelentkezés az újonnan regisztrált felhasználóval.
Get Current User (me)	Az aktuálisan bejelentkezett felhasználó adatainak lekérése.
Get Current User (me) - unauthorized	Próbálkozás a felhasználói adatok lekérésére hitelesítés nélkül (401-es hiba várt).
Get Public User by Id	Nyilvános felhasználói profil lekérése azonosító alapján (nem igényel hitelesítést).
Get Public User - not found	Próbálkozás nem létező felhasználó lekérésére (404-es hiba várt).
Update Current User (me)	A bejelentkezett felhasználó profilképének és nevének frissítése.
Login - invalid credentials	Bejelentkezés próbája érvénytelen jelszóval (401-es hiba várt).
Login - bad request (missing password)	Bejelentkezés próbája jelszó megadása nélkül (400-as hiba várt).
List Workspaces	A felhasználó munkaterületeinek listázása.
List Workspaces - unauthorized	Munkaterületek listázása hitelesítés nélkül (401-es hiba várt).
Create Workspace	Új munkaterület létrehozása.
Create Workspace - bad request (missing name)	Munkaterület létrehozása név nélkül (400-as hiba várt).
Get Workspace	Egy konkrét munkaterület adatainak lekérése azonosító alapján.
Update Workspace	Egy munkaterület nevének frissítése.

Get Workspace - not found	Próbálkozás nem létező munkaterület lekérésére (404-es hiba várt).
Create Tag	Új címke létrehozása egy munkaterületen belül.
List Tags	Egy munkaterülethez tartozó címkék listázása.
Get Tag	Egy konkrét címke adatainak lekérése.
Update Tag	Egy címke nevének és színének frissítése.
Create Tag - bad request (invalid color)	Címke létrehozása érvénytelen színinformációval (400-as hiba várt).
Get Tag - not found	Próbálkozás nem létező címke lekérésére (404-es hiba várt).
Create Note (Text)	Új szöveges jegyzet létrehozása a munkaterületen.
Create Note (URL)	Új URL típusú jegyzet létrehozása.
List Notes	A munkaterület összes jegyzetének listázása.
Get Note	Egy konkrét jegyzet részletes adatainak lekérése.
Update Note	Egy jegyzet címének, tartalmának és pozíciójának frissítése.
Create Note - bad request (missing name/kind)	Jegyzet létrehozása kötelező mezők nélkül (400-as hiba várt).
Get Note - not found	Próbálkozás nem létező jegyzet lekérésére (404-es hiba várt).
Attach Tag to Note	Címke hozzárendelése egy jegyzethez.

Get Note-Tag Association	Ellenőrzés, hogy a címke hozzá van-e rendelve a jegyzethez.
Create Note Relation	Kapcsolat létrehozása két jegyzet között.
Get Note Relation	Egy jegyzetkapcsolat adatainak lekérése.
Update Relation	Egy jegyzetkapcsolat nevének frissítése.
Create Relation - bad request (missing fields)	Kapcsolat létrehozása hiányos adatokkal (400-as hiba várt).
Get Relation - not found	Próbálkozás nem létező kapcsolat lekérésére (404-es hiba várt).
Detach Tag from Note	Címke eltávolítása egy jegyzetről.
Delete Note Relation	Jegyzetek közötti kapcsolat törlése.
Delete Note	Jegyzet törlése.
Delete Tag	Címke törlése.
Delete Workspace	Munkaterület törlése.
List Organizations	A felhasználó szervezeteinek listázása.
List Organizations - unauthorized	Szervezetek listázása hitelesítés nélkül (401-es hiba várt).
Create Organization	Új szervezet létrehozása.
Create Organization - bad request (missing name)	Szervezet létrehozása név nélkül (400-as hiba várt).
Get Organization	Egy szervezet adatainak lekérése.
Get Organization - not found	Próbálkozás nem létező szervezet lekérésére (404-es hiba várt).
Get Organization - unauthorized	Szervezet lekérése hitelesítés nélkül (401-es hiba várt).
Update Organization	Egy szervezet nevének frissítése.

Update Organization - not found	Nem létező szervezet nevének frissítése (404-es hiba várt).
Get Organization Workspaces	Egy szervezet munkaterületeinek listázása.
Get Organization Workspaces - unauthorized	Szervezeti munkaterületek listázása hitelesítés nélkül (401-es hiba várt).
Get My Workspace Access	A felhasználó saját hozzáférési szintjének lekérése egy munkaterülethez.
Get My Workspace Access - unauthorized	Saját hozzáférés lekérése hitelesítés nélkül (401-es hiba várt).
List Workspace Access	Az összes hozzáférés listázása egy munkaterülethez (Admin jogosultság szükséges).
Grant Workspace Access	Hozzáférés megadása egy másik felhasználónak a munkaterülethez.
Update Workspace Access	Egy meglévő munkaterület-hozzáférés módosítása.
Revoke Workspace Access	Hozzáférés visszavonása a munkaterülettől.
Get My Organization Access	A felhasználó saját hozzáférési szintjének lekérése egy szervezethez.
Get My Organization Access - unauthorized	Saját szervezeti hozzáférés lekérése hitelesítés nélkül (401-es hiba várt).
List Organization Access	Az összes hozzáférés listázása egy szervezethez.
Grant Organization Access	Hozzáférés megadása a szervezethez.
Revoke Organization Access	Hozzáférés visszavonása a szervezettől.

Get My Invitations	A felhasználónak küldött összes meghívó lekérése.
Get My Invitations - unauthorized	Meghívók lekérése hitelesítés nélkül (401-es hiba várt).
Create Workspace Invitation	Meghívó létrehozása egy munkaterülethez.
Set Organization Workspace Owner	Szervezet beállítása egy munkaterület tulajdonosaként.
Remove Organization Workspace Owner	Munkaterület eltávolítása a szervezet tulajdonából.
Delete Organization	Szervezet törlése.
Logout	Kijelentkezés az alkalmazásból (token érvénytelenítése).
Logout - missing header	Kijelentkezés próbája token nélkül (401-es hiba várt).
Logout - invalid token	Kijelentkezés próbája érvénytelen tokennel (401-es hiba várt).
WebSocket.http	
WS-001: Successful Auth + SessionInfo + Ping/Pong	Sikeres WebSocket csatlakozás, hitelesítés, munkamenet-információk fogadása és szívverés ellenőrzése.
WS-002: Lock and Unlock Actions	Jegyzet zárolási és feloldási műveletek ellenőrzése WebSocket-en keresztül.
WS-003: Client Ping Test	Kliens oldali Ping üzenet és szerver válasz ellenőrzése.

Frontend

Bevezetés

A Graphlet felhasználó felülete TypeScript nyelven íródott. Azon belül a [React](#) könyvtár segítségével.

Felépítés és szerkezet

Mappa szinten

A frontend projekt mappája a „**backend/Frontend/Graphlet**” mappában található. Ott, az „**src**” mappában vannak a főbb oldalak a „**pages**” mappában, React komponensek a „**components**” mappában, és egyéb megjelenő elemek az „**assets**” mappában.

Kód szinten

App.tsx és bejelentkezés

A frontend fő oldala az App.tsx, ami csak egy RouterProvider-t tartalmaz semmi mást, ami csak az alap útvonalakat tartalmazza, és az alapján lehet navigálni. Az útvonalak a „**components/router/router.tsx**” fájlban találhatóak. Ebbe be vannak importálva a különböző oldalak komponensei, és mindnek megva a maga elérési útvonala. Az alapútvonalra való navigáláskor automatikusan a loginra kerülünk. A beállításoknak pedig vannak leszármazott elemei, amik a különböző beállítások saját oldalai. A navigáláshoz a React által behozott Navigate elemeket használjuk.

Az alap elérési útvonal „/” a login oldalra navigál. Ez a pages mappában van. Ezen az oldalon lehet bejelentkezni.

Minden oldal és komponens elején be vannak importálva a szükséges komponensek, és fájlok. Importálás után definiálva van egy szabály az e-mail cím felépítésére. Létre vannak hozva változók, amik tárolják a bejelentkezési folyamat állapotát egy useState-ben.

A login funkció -nem összetévesztendő a Login-nal, mert az a komplett komponens- felel a bejelentkezésért. Először megszerzi az adatokat a megfelelő helyről a css osztálynév alapján, majd ellenőrzi azokat. Utána kérést indít a backend felé, amiben elküldi az e-mail címet és a jelszót és a választ pedig elmenti, később json formátumba konvertálja azt. Amennyiben a válasz kódja 200, akkor a visszaküldött tokent elmenti a böngészőbe, és sikeresen bejelentkezünk és átkerülünk a Workspaces oldalra. Ellenkező esetben hibaüzenet jelenik meg, és nem tudunk belépni.

Az oldalon van még egy link, amivel átnavigálhatunk a regisztrációs oldalra, ahol létrehozhatunk egy új fiókot.

Regisztráció

A Login oldalról lehet idejutni. Itt lehet új felhasználói fiókot létrehozni. Kell egy e-mail cím, egy felhasználónév, és egy jelszó. Itt is van e-mail cím formai ellenőrzés, hibaüzenet, amik a komponens elején beállításra kerülnek. A registerUser funkcióban a loginhoz képest még egy username is van. A bevitt értékek létezése, és formai követelményeknek való megfelelése itt is ellenőrizve van hasonlóképp, mint a bejelentkezésnél. Ezután indul a kérés a szerver felé a megfelelő végpontra. Hiba esetén megjelenik a hibaüzenet, és nem történik átirányítás. Amennyiben sikeres a regisztráció, arról is kapunk visszajelzést, és a weboldal átirányít minket a bejelentkezési felületre, ahol az újonnan létrehozott fiókunkba tudunk bejelentkezni. Továbbá van még egy hivatkozás az oldal alján, a login oldalra tudunk visszamenni.

Workspaces oldal

Erről az oldalról érhetők el az oldal fő funkciói. Itt vannak a jegyzetek, innen érhetők el a beállítások, a kijelentkezés.

Ez az oldal több segédkomponenst is használ, amik később részletezve lesznek.

Az első funkció, a getWorkspaces felel azért, hogy amikor a felhasználó megnyitja az oldalt, akkor a szerverről betöltse a már létrehozott workspace-eket. Egy promise-szal tér vissza, amiben egy Workspaceket tartalmazó tömb van. Kérés indul a szerverhez, és már a korábban elmentett autentikációs tokent is használja, egyrésztől, hogy megbizonyosodjon a program arról, hogy a felhasználónak van-e jogosultsága, másrésztől pedig, hogy azonosítani tudja a felhasználót, és a megfelelő workspace-eket adja oda neki. Ha sikerrel tér vissza a szerver, akkor átalakítja a választ, és

visszaadja a tömböt. Ha üres a tömb, akkor megjelenik egy szöveg, hogy nincs még létrehozott workspace. Hiba esetén hibaüzenet jelenik meg.

A komponens fő részében definiálva vannak értékek: maguk a workspace-k, a betöltés állapota, az aktuális hiba, az egyéb opciók megjelenési állapota, az új workspace létrehozása ablak megjelenési állapota, a keresőmező tartalma, megnyitandó workspace url-je, és az url-ben lévő beállítások, amik később kellenek.

A `loadWorkspaces` metódus betölti a szerverről lekért workspace-eket a változóba, aminek a tartalma dinamikusan jelenik meg. A következő kisebb szekció, a `useEffect` azt a célt szolgálja, hogy amikor betölt az oldal, akkor egyből betöltse a workspace-eket. Ezután következik néhány eseménykezelő függvény:

`handleOtherOptionsClick`: az egyéb opciók menü megjelenéséért szolgál.

`handleCreateNewClick`: az új workspace létrehozása menü megjelenéséért szolgál.

`handleCloseCreatingNew`: az új workspace létrehozása menü bezárásáért szolgál.

`handleRename`: a workspace átnevezése opció megjelenítése

`handleDelete`: workspace törlése opció

`filteredWorkspaces`: ez egy változó, amiben a keresés eredményi találhatóak. Ha üres, akkor szimplán nem jelenik meg semmi.

`openWorkspaceInNewTab`: ha rákattintunk egy workspace-re, hogy megnyissuk, akkor ez az a függvény, ami a jó linkre visz minket, és új oldalon nyitja meg.

Az oldalon vannak statikus részek: a felső sávban, de a workspace-ek (`WorkspacePreview` komponens) megjelenítése és elrendezése teljesen dinamikus, mert mindegyiknek egyedi azonosítója van.

WorkspacePreview

Ez a komponens tartalmaz egy interface-t, ami alapján épül fel. Ezután a `WorkspacePreview` az interface alapján létrejön. A változók/állapotok:

- `showMenu`: a menü megjelenésének aktuális állapota – bool
- `confirmMode`: egy művelet megerősítésének módja ami lehet átnevezés, törlés, vagy null.
- `renameValue`: az átnevezéskor megadott új név - szöveg

- isSubmitting: állapot, ami addig igaz, amíg nem érkezik vissza a szervertől a válasz, miután módosítási vagy törlési kérelmet indítottunk - bool
- error: az aktuális hibaüzenet – szöveg, vagy null lehet

handleRenameConfirm funkció: Az átnevezésért felel. Bemeneti paramétere az új név. A kérés a szerver felé a workspace id-ja alapján történik, és a body tartalmazza az új nevet. Amennyiben valami hiba történik, akkor a hibaüzenet metváltozik

Other options

Ez a komponens a Workspaces oldal jobb felső sarkában foglal helyet, amennyiben megnyomjuk a három pontot. Ez egy kicsi felugró ablak, ahonnan elérhetőek egyéb funkciók, például a kijelentkezés. A kijelentkezés gombra kattintva a program eltávolítja a böngészőből az egyedi tokent, és visszairányít bennünket a bejelentkezés oldalra.

Options Oldal:

Szervezet (Szervezetkezelés)

Szervezet létrehozása: új szervezet nevének megadásával.

- **Saját szervezetek listája:** minden szervezet külön „kártyán” jelenik meg.
- **Munkaterületek kezelése a szervezeten belül:**
 - munkaterület hozzáadása a szervezethez
 - munkaterület eltávolítása a szervezetből
- **Tagok / jogosultságok kezelése (csak Tulajdonos esetén):**
 - taglista megtekintése
 - tag jogosultsági szintjének módosítása (pl. Olvasás / Írás / Tulajdonos – a felületen elérhető opciók szerint)
 - tag eltávolítása
- **Meghívás a szervezetbe (csak Tulajdonos esetén):**
 - meghívás másik felhasználó felhasználói azonosítója (User ID) alapján

Jogosultsági logika (felületi szinten): Egyes gombok (pl. törlés, meghívás, kezelés) csak Tulajdonos esetén aktívak, egyébként le vannak tiltva.

Profilbeállítások

A profil oldalon:

- Megtekintheted a saját adataidat (felhasználónév, e-mail, profilkép URL stb.).
- Módosíthatod a profiladataidat (pl. felhasználónév / e-mail / profilkép URL).
- Elérhető egy **Felhasználói azonosító másolása** gomb: ez különösen hasznos, mivel több helyen (pl. szervezeti meghívónál) azonosítóval hivatkoztok a felhasználókra.
- Opcionálisan elérhető egy „felhasználó keresése azonosító alapján” funkció is (nyilvános profiladatok gyors ellenőrzésére).

Megosztott elemek

Ez a nézet a szervezeti megosztásokat segít áttekinteni:

- Szervezetben belül megosztott munkaterületek listázása.
- A kezelési műveletek (pl. megosztás visszavonása) alapvetően Tulajdonoshoz kötöttek.
- Ha nincs megosztott munkaterület, a lista üres állapotot jelenít meg.

Meghívók

Itt látod a függőben lévő meghívásaidat (pl. munkaterületre vagy szervezetbe szólóak).

Funkciók:

- Meghívók listázása
- **Elfogadás:** elfogadod a meghívást (hozzáférést kapsz)
- **Elutasítás:** elutasítod a meghívást (eltűnik a listából)
- Ha nincs függőben lévő meghívó, „No pending invitations” üzenet jelenik meg.

Frontend tesztek

Automata tesztelés

Az automata tesztelőprogramot c# nyelven írtuk meg Selenium rendszerben. Két fő mappája van a tesztprogramnak: az oldalak felépítését tartalmazó mappa (page model) és a tesztek tartalmazó mappa (tests). A page model fájlok tartalmazzák egy-egy oldal felépítését, és egyéb kisegítő funkciókat, amelyekre a tesztelőszoftver futása során szükség van.

A program 76 tesztet fed le, mindhez tartozik manuális teszt is.

test_id	step	step name	állapot
login			
1	1	Átirányítás a register oldalra	ok
2	1	Üres adattal való bejelentkezés	ok
	2	hibaüzenet keresés	ok
3	1	Üres emaillel való bejelentkezés	ok
	2	hibaüzenet keresés	ok
4	1	üres jelszóval való bejelentkezés	ok
	2	hibaüzenet keresés	ok
5	1	hibás adatokkal való bejelentkezés	ok
	2	hibaüzenet keresés	ok
6	1	hibás emaillel való bejelentkezés	ok
	2	hibaüzenet keresés	ok
7	1	hibás jelszóval való bejelentkezés	ok
	2	hibaüzenet keresés	ok
8	1	helyes adatokkal való bejelentkezés	ok
	2	workspaces oldalon vagyunk-e	ok

register			
1	1	visszatérés a login oldalra	ok
2	1	üres adatokkal való regisztráció	ok
	2	hibaüzenet keresés	ok
3	1	üres email	ok
	2	hibaüzenet keresés	ok
4	1	üres jelszó	ok
	2	hibaüzenet keresés	ok
5	1	egyéb hibás formátum, pl email	ok
	2	hibaüzenet keresés	ok
6	1	rossz email	ok
	2	hibaüzenet keresés	ok
7	1	rövid jelszó	ok
	2	hibaüzenet keresés	ok
8	1	jó adatok	ok
	2	át navigálás a login oldalra	ok
9	1	megjelenő üzenet sikeres regisztráció esetén	ok

settings			
1	1	checkinh heading text	ok
2	1	checking checkbox visiblity	ok
3	1	checking apperance button vorking	ok
4	1	checking profile button vorking	ok
5	1	checking shared button vorking	ok
6	1	checking subscription button vorking	ok
7	1	working go back button	ok
	2	checking url	ok
8	1	nav to setting root --> to apppearance	ok
	2	checking url	ok
9	1	checking profile page heading	ok
10	1	checking setting accessibility - url	ok
11	1	shared page heading text	ok
12	1	subscription page heading	ok

workspaces			
1	1	cancel should not delete the workspace	ok
2	1	cancel rename should keep the name	ok
3	1	clear in searchbar should return every workspace	ok
4	1	created workspace should appear	ok
5	1	created workspace should have the correct name	ok
6	1	creating with empty name should keep create button	ok
7	1	deleting workspace should remove it from the list	ok
8	1	goto settings should open settings page	ok
9	1	logout should log the user out and return to login page	ok
10	1	create new dialog should show when clicked	ok
11	1	opening other options should show the panel	ok
12	1	clicking on workspace should open in new tab	ok
13	1	renaming workspace should update the name	ok
14	1	search with existing text should return matching	ok
15	1	search with unexisting text should return nothing	ok

workspace			
1	1	check new note if it has any default content	ok
2	1	check new note if it has any default header	ok
3	1	added notes should appear - adding 3	ok
	2	check count	ok
4	1	add note button should be visible	ok
5	1	note count should increase when new note added	ok
	2	checking note count	ok
6	1	cancel edit exits edit mode	ok
7	1	cancel edit should not modify the content	ok
8	1	cancel edit should not modify the title	ok
9	1	the canvas should be visible	ok
10	1	the closing button should be visible	ok
11	1	close button should close the page	ok
12	1	the close button should close the page	ok
13	1	deleting notecard should remove it from dom	ok
14	1	deleting notecard should remove it from canvas	ok
15	1	deleting all notes should result an empty canvas	ok
16	1	double clicking on a note should enable edit mode	ok
17	1	textarea should contain the saved text in edit mode	ok
18	1	textarea should contain the saved title in edit mode	ok
19	1	cancel edit button should be visible	ok
20	1	edit mode should show input fields	ok
21	1	save button should be visible	ok
22	1	title input should be visible in edit mode	ok
23	1	the error div should be present in dom	ok
24	1	moved note should keep the position after reload	ok
25	1	moved note should not revert the moving	ok
26	1	moving note should change the position	ok
27	1	the notecard should show content	ok
28	1	notecard should show delete button	ok
29	1	notecard should show delete title	ok
30	1	saving should exit edit mode	ok
31	1	save should update note content	ok
32	1	save should update title content	ok
33	1	toolbar should be visible	ok

Manuális tesztelés

Manuális tesztek

Login

Teszt eset 1: A register oldalra való átirányítás sikeresen megtörtént, a login oldalra való visszatérés hibátlan volt, és minden navigációs funkció rendben működött.

Teszt eset 2: Üres adattal történő bejelentkezés során a rendszer hibátlanul jelezte a hiányzó adatokat, a hibaüzenet megjelenése sikeres volt, és a felhasználó megfelelően tájékoztatva lett.

Teszt eset 3: Üres email mezővel történő bejelentkezés sikeresen megtörtént, a rendszer hibátlanul jelezte a hiányzó emailt, és a hibaüzenet helyesen jelent meg.

Teszt eset 4: Üres jelszóval történő bejelentkezés során a rendszer hibátlanul jelezte a hiányzó jelszót, a hibaüzenet megjelenése rendben zajlott, és minden funkció megfelelően működött.

Teszt eset 5: Hibás adatokkal történő bejelentkezés sikeresen megtörtént, a rendszer helyesen jelezte az adatokat, a hibaüzenet megjelenése hibátlan volt, és minden visszajelzés pontosan jelent meg.

Teszt eset 6: Hibás email cím megadásakor a rendszer megfelelően jelezte a hibát, a hibaüzenet hibátlanul jelent meg, és a felhasználó minden esetben értesítést kapott a problémáról.

Teszt eset 7: Hibás jelszó megadásakor a rendszer rendben jelezte a hibát, a hibaüzenet megjelenése sikeres volt, és a felhasználói interakciók zavartalanul működtek.

Teszt eset 8: Helyes adatokkal történő bejelentkezés sikeresen megtörtént, a felhasználó a workspaces oldalon hibátlanul jelent meg, és minden navigációs és megjelenítési funkció rendben működött.

Register

Teszt eset 1: A login oldalra való visszatérés sikeresen megtörtént, minden navigáció hibátlanul működött, és a felhasználó rendben visszakerült a kezdőoldalra.

Teszt eset 2: Üres adatokkal történő regisztráció során a rendszer hibátlanul jelezte a hiányzó mezőket, a hibaüzenetek rendben jelentek meg, és a felhasználó értesítése sikeres volt.

Teszt eset 3: Üres email mezővel történő regisztráció során a rendszer hibátlanul jelezte a hiányzó emailt, a hibaüzenet rendben megjelent, és minden funkció megfelelően működött.

A többi manuális teszt elérhető a [Manuális tesztek.docx](#) fájlban

Felhasználói dokumentáció

A probléma leírása és a szoftver célja

A modern információ-túlkínálat korában a hagyományos, lineáris jegyzetelési módszerek (mint egy egyszerű Word dokumentum vagy egy fizikai füzet) gyakran elégtelenek a komplex, összefüggő adatok megértéséhez. Amikor egy projekttervet, egy kutatási anyagot, vagy egy cég szervezeti felépítését próbáljuk átlátni, a pusztá szöveg nem adja vissza az elemek közötti kapcsolatrendszer.

A probléma:

A meglévő vizuális eszközök (pl. folyamatábra-készítők) túl merevek, míg a jegyzetalkalmazások nem nyújtanak térbeli, gráf-alapú áttekintést. Ráadásul a csapatmunkát, a jogosultságok finomhangolását (ki mit láthat és szerkeszthet) kevés eszköz kezeli kellő rugalmassággal.

A Graphlet megoldása:

A Graphlet egy kollaboratív, vizuális munkaterület (Workspace) kezelő platform. Lényege, hogy a felhasználók "Jegyzeteket" (Notes) hozhatnak létre egy végtelen virtuális vásznon, amelyeket szabadon összeköthetnek (Relations), és Címkékkel (Tags) láthatnak el. A szoftver célja, hogy a gondolatokat és adatokat ne listákban, hanem hálózatokban (gráfokban) reprezentálja, megkönnyítve ezzel az agilis csapatok, kutatók és menedzserek mindennapi munkáját, beépített szervezet-kezeléssel (Organization) és megosztási beállításokkal.

2.2. Szerepkörök bemutatása

A Graphlet rugalmas, hierarchikus jogosultsági rendszert alkalmaz, amely egyaránt alkalmas egyéni felhasználásra és nagyvállalati, több-bérlős (multi-tenant) környezetbe.

1. Regisztrált Felhasználó (User)

- Leírás: A rendszer alapvető aktora. Mindenki, aki sikeresen regisztrál és belép az alkalmazásba, ebbe a kategóriába kerül.

- **Képességek:** * Saját Munkaterületek (Workspaces) létrehozása, átnevezése és törlése.
 - A saját profiladatok (Profile Settings) és profilkép (picurl) kezelése.
 - Teljes körű olvasási és írási (CRUD) jog a saját gráfjain (Jegyzetek, Címkék, Kapcsolatok felvétele).
 - Megosztási beállítások (Sharing settings) menedzselése: meghívhat másokat a saját munkaterületére.

2. Szervezeti Adminisztrátor (Organization Admin / Owner)

- **Leírás:** A Graphlet támogatja a Szervezetek (Organization) létrehozását. Egy szervezet egy céget vagy nagyobb projektcsapatot jelöl, amely alá több Munkaterület és Felhasználó is tartozhat.
- **Képességek:**
 - A szervezet nevének és adatainak kezelése.
 - Csoportok (Groups) létrehozása a tagok tömeges kezeléséhez (pl. "Fejlesztők", "Marketing").
 - Tagok menedzselése (Member management): új kollégák felvétele a szervezetbe, jogosultságok kiosztása, vagy a hozzáférés visszavonása.
 - Átfogó rálátás a szervezet alá tartozó összes Munkaterületre.

3. Publikus / Meghívott Felhasználó (Public User / Guest)

- **Leírás:** Olyan felhasználó, aki egy adott Munkaterülethez csak korlátozott (pl. csak olvasható) hozzáférést kapott egy megosztási linken keresztül, vagy egy Szervezet tagja korlátozott jogokkal.
- **Képességek:**
 - A gráfok megtekintése, a vásznon való navigáció (nagyítás, kicsinyítés).
 - Más felhasználók publikus adatainak (Név, Email, Profilkép) megtekintése a taglistában.
 - *Nem* hozhat létre új Jegyzeteket, és nem módosíthatja a Kapcsolatokat.

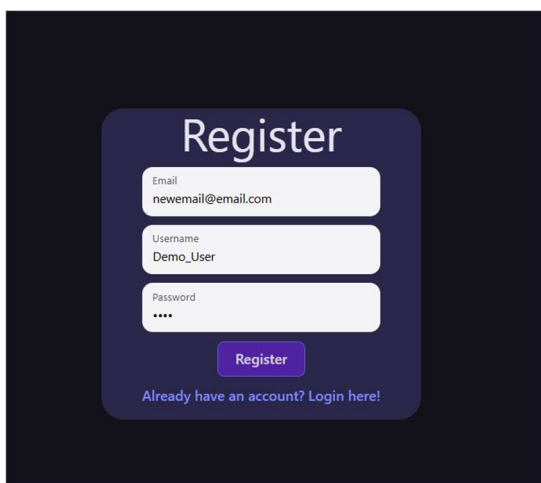
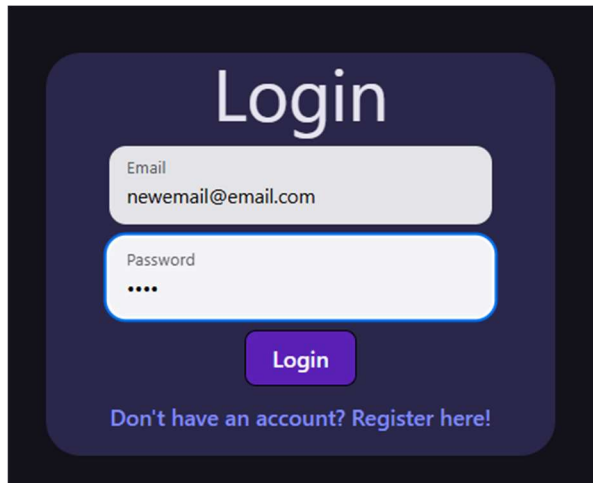
2.3. Felhasználói esetek (Use Cases) és Üzleti logikák

Az alábbiakban lépésről lépésre bemutatjuk a rendszer legfontosabb funkcióit életszerű szituációkon keresztül.

1. Eset: Bevezetés és Profil beállítása (Onboarding)

Anna, egy projektmenedzser most hallott először a Graphletről, és szeretné elkezdni használni a csapata számára.

1. **Regisztráció:** Anna megnyitja a nyitóoldalt, és a "Register" opciót választja. Megadja az email címét és egy jelszót. A rendszer létrehozza az UUID-ját.
2. **Belépés:** A sikeres regisztráció után a "Login" képernyőn hitelesíti magát. Az API ellenőrzi az adatokat, és betölti neki a fő vezérlőpultot (MyWorkspaces).
3. **Profil testreszabása:** A vezérlőpulton a "Settings" (Beállítások) ágon belül kiválasztja a "Profile Settings" menüpontot. Feltölt egy profilképet (amit a rendszer a picurl mezőben tárol el), beállítja a teljes nevét, majd elmenti.

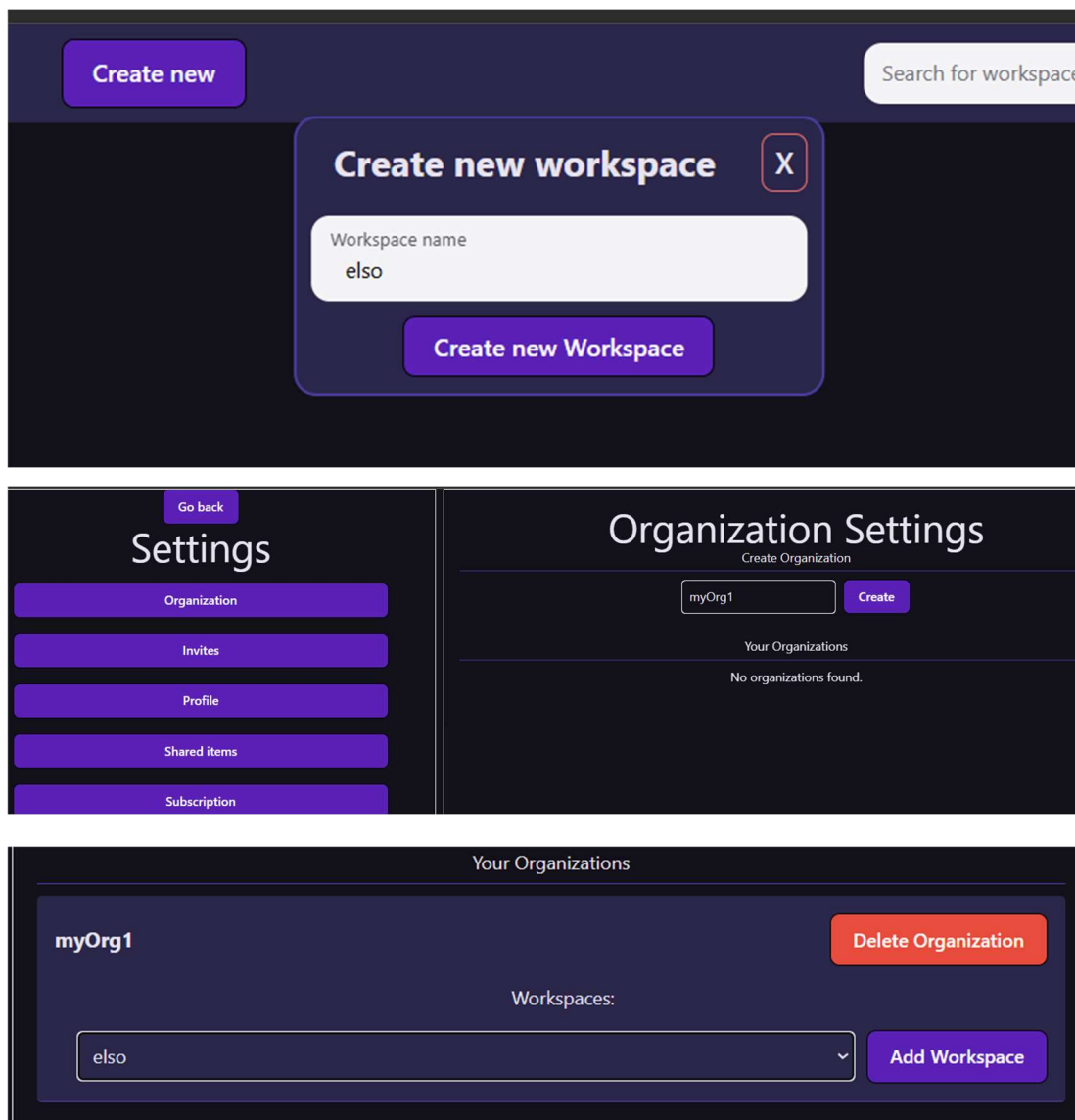
A screenshot of the 'Register' form. It has a dark blue background with a white rounded rectangle containing the form fields. The title 'Register' is at the top. There are three input fields: 'Email' with the value 'newemail@email.com', 'Username' with the value 'Demo_User', and 'Password' with four dots. Below the fields is a blue 'Register' button. At the bottom, there is a link: 'Already have an account? Login here!'.A screenshot of the 'Login' form. It has a dark blue background with a white rounded rectangle containing the form fields. The title 'Login' is at the top. There are two input fields: 'Email' with the value 'newemail@email.com' and 'Password' with four dots. Below the fields is a blue 'Login' button. At the bottom, there is a link: 'Don't have an account? Register here!'.

2. Eset: Csapatmunka és Munkaterület (Workspace) indítása

Anna profilja kész, most létre kell hoznia a helyet, ahol a csapata dolgozhat az új szoftver architektúráján.

1. **Munkaterület létrehozása:** A "MyWorkspaces" felületen az "Új Munkaterület" gombra kattint. Megadja a nevet: *Q3 Szoftver Tervezés*.
2. **Szervezeti beállítások:** Mivel ezt a céges csapattal akarja megosztani, a "Workspace" menüből átlép a "Member management" (Tagok kezelése) felületre.

3. **Tagok meghívása:** Keresést indít a kollégája, "Bence" email címére. A rendszer a Public User adatok alapján kilistázza Bencét. Anna hozzáadja őt a munkaterülethez szerkesztői (Editor) joggal. Bence a saját "MyWorkspaces" felületén azonnal látni fogja a *Q3 Szoftver Tervezés* projektet a megosztott elemek között.

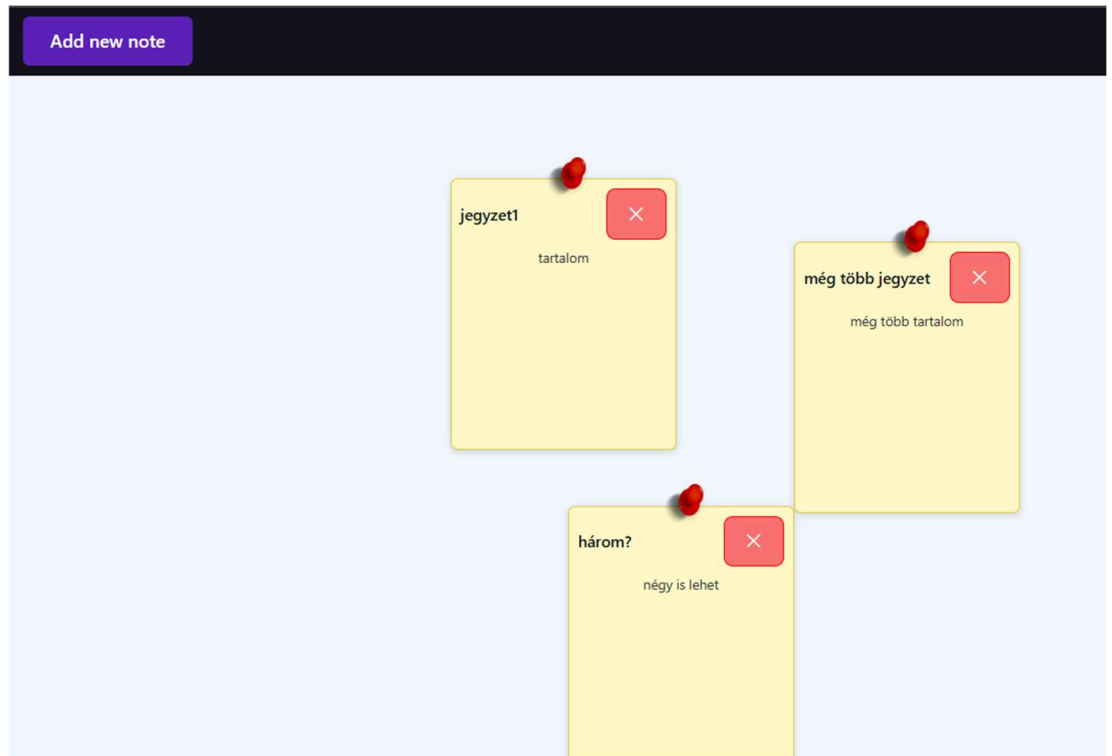


3. Eset: A Vizuális Gráf Építése (Jegyzetek és Címkék)

Bence belép a munkaterületre, hogy elkezdje felvázolni az ötleteit, ekkor a rendszer betölti a vizuális vásznat (RenderFrame).

1. **Jegyzet (Note) létrehozása:** Bence rá kattint az új jegyzet gomra. Létrejön egy új Jegyzet. Megjelenik egy új jegyzet. Beírja a címet: *Frontend Szerver*, és az értéket: *React alapú SPA alkalmazás*.

2. **Kategorizálás Címkékkel (Tags):** Hogy átláthatóbb legyen a rendszer, Bence a "Settings" -> "Tags" menüben létrehoz egy új címkét: Név: *Kritikus komponens*, Szín: *Piros*.
3. Ezután rákattint a *Frontend Szerver* jegyzetre, és hozzárendeli ezt a piros címkét (Note-tag association). A jegyzet sarka azonnal pirosra színeződik.

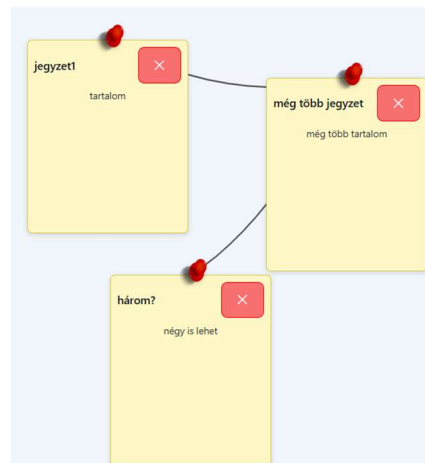
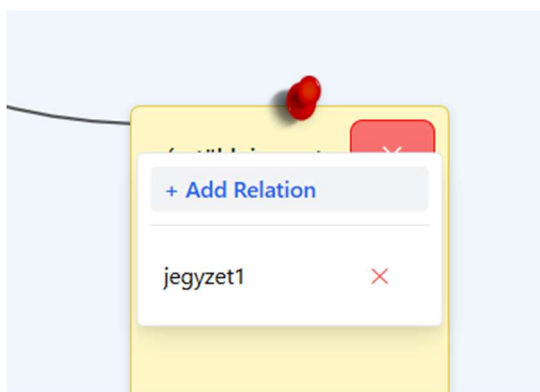


4. Eset: Kapcsolatok (Relations) kialakítása

Az igazi varázslat akkor történik, amikor a jegyzetek hálózattá állnak össze.

1. Bence létrehoz egy harmadik jegyzetet: *Adatbázis (PostgreSQL)*.
2. **Összekötés:** Az egeret a *Frontend Szerver* jegyzeten tartva, a megjelenő csatlakozási pontról (handle) húz egy vonalat az *Adatbázis* jegyzetig.
3. **Kapcsolat elnevezése:** A vonal (Relation) létrejön. Bence rákattint magára a vonalra, és ad neki egy nevet: *"Adatokat kér le REST API-n"*.

4. **A végeredmény:** A rendszer az adatbázisban létrehozott egy NoteRelation bejegyzést, ami összeköti a két egyedi azonosítót. A vásznon egyértelmű, vizuálisan értelmezhető folyamatábra alakult ki, amelyet Anna és Bence valós időben, közösen tudnak tovább fejleszteni.



Összefoglalás

A **graphlet** projekt egy iskolai vizsgaremek (záróprojekt).

Főbb jellemzői:

- **Technológiai stack:** A rendszer egy **ASP.NET** alapú backendből és egy **React** alapú frontendből áll. A kódbázis nagyrészt **C#** (68%) és **TypeScript** (26%) nyelveken íródott.
- **Futtatás és telepítés:** A projekt konténerizált, így **Docker** segítségével nagyon egyszerűen elindítható.

Hogyan lehet elindítani? A futtatáshoz csak le kell klónozni a repo-t, és a **Docker Compose** segítségével elindítani a szolgáltatásokat:

1. `git clone` (a tároló letöltése)
2. `cd graphlet` (belépés a mappába)
3. `docker compose -f compose.yaml up --build` (a konténerek felépítése és indítása)
4. Ezt követően az alkalmazás a böngészőből elérhető a `http://localhost:5173` címen.

Összességében ez egy modern webes technológiákat (.NET + React + Docker) felvonultató vizsgamunka.

Kitekintés

A Graphlet jelenlegi formájában egy stabil, működőképes alapot nyújt, amelyre számos irányban lehet tovább építeni. Az alkalmazás láthatóságának és elérhetőségének növelése érdekében érdemes lenne egy dedikált **landing page** készítése, amely bemutatja a projekt funkcióit és felhasználási lehetőségeit. A felhasználói élmény gazdagítása céljából tervbe vehető **több jegyzettípus** bevezetése (például checklist, táblázat vagy rajzalapú jegyzetek), valamint **többnyelvű felület** kialakítása, hogy az alkalmazás szélesebb közönség számára is elérhető legyen. A személyre szabhatóság jegyében különböző **vizuális témák** – például világos és sötét mód – támogatása is megvalósítható lenne.

A közösségi és biztonsági funkciók terén szintén jelentős fejlesztési potenciál rejlik a projektben. Egy beépített **privát csevegő** funkció lehetővé tenné a felhasználók közötti közvetlen kommunikációt az alkalmazáson belül. Az autentikáció modernizálása érdekében **OAuth-alapú bejelentkezés** integrálása is célszerű lenne, amely lehetővé tenné a felhasználók számára, hogy meglévő fiókjaikkal – például Google- vagy GitHub-fiókkal – jelentkezzenek be, ezzel egyszerűsítve a regisztrációs folyamatot és növelve a rendszer biztonságát.

Irodalmi jegyzék

[React dokumentáció](#)

[W3Schools](#)

[StackOverflow](#)

[Docker dokumentáció](#)

[YouTube](#)

[Microsoft docs](#)